

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ**

Кафедра програмної інженерії

ЗВІТ

з лабораторної роботи № 1

«НАЛАШТУВАННЯ ПРОФЕСІЙНОГО СЕРЕДОВИЩА РОЗРОБКИ:

IDE, Git, AI Copilot»

з дисципліни

«Основи програмної інженерії»

Рапія Рустама Сергійовича ПЗПІ-25-5

В. І. Каук, В. О. Гребенюк

Харків 2026

Метою роботи є набуття практичних навичок з налаштування професійного середовища розробки програмного забезпечення, а також оволодіння базовими операціями системи контролю версій Git та платформи GitHub. Особлива увага приділяється ознайомленню з можливостями AI-асистентів кодування для підвищення продуктивності програміста.

В ході виконання роботи було поставлено та вирішено наступні завдання:

1. встановлення та налаштування IDE Visual Studio Code з відповідними розширеннями для мови програмування C#;
2. практичне оволодіння послідовністю операцій Git: init, add, commit, push, branch, merge;
3. створення, наповнення та оформлення Pull Request на платформі GitHub;
4. використання AI-асистента (GitHub Copilot) для генерації функціональних фрагментів коду за текстовим описом для проєкту Empathix.

Назва проєкту: Empathix

Empathix — це інтелектуальний веб-помічник, призначений для формування коректних та емпатичних повідомлень у складних життєвих або робочих ситуаціях. Система допомагає користувачам аналізувати емоційне забарвлення тексту, уникати токсичності та автоматизувати підбір доцільних професійних виразів.

Ключовий функціонал:

1. Генерація відповідей за допомогою AI (OpenAI API) з урахуванням обраного тону (професійний, дружній, м'який).
2. Контроль довжини повідомлень та лімітів запитів для різних категорій користувачів.

Хід виконання роботи:

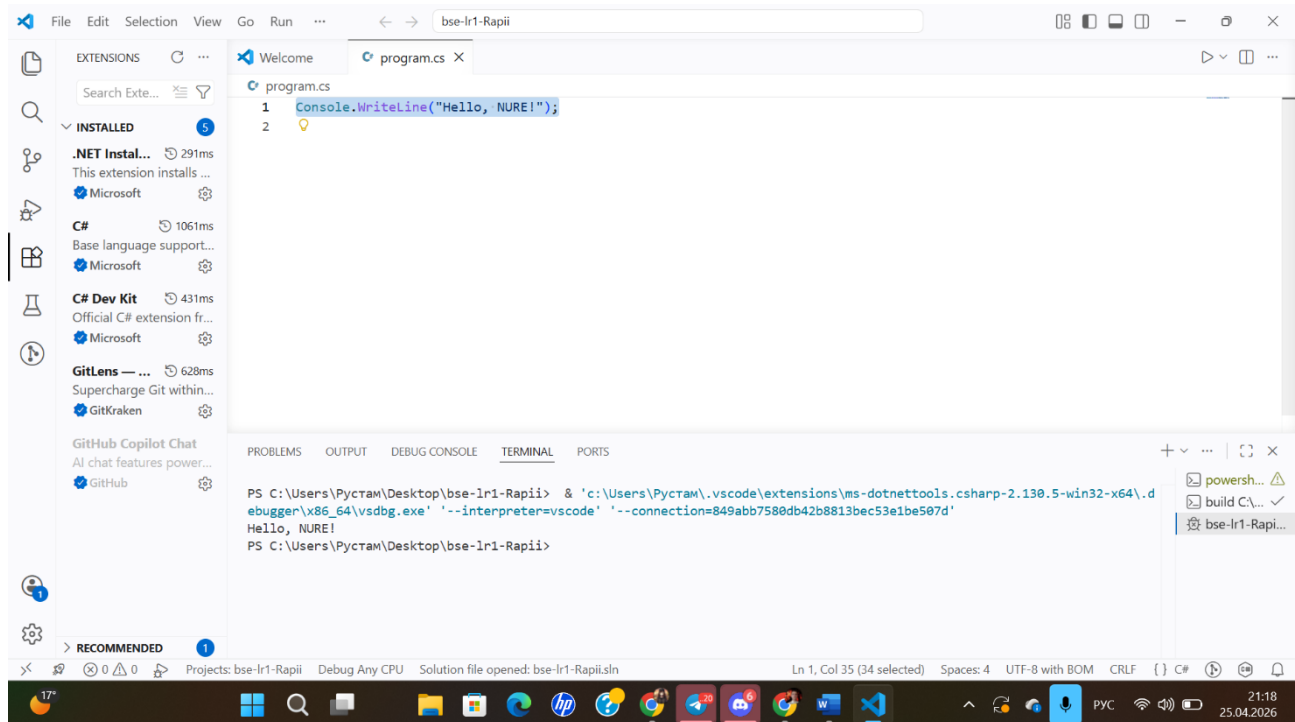


Рисунок 4.1 — Встановлення розширень для розробки на C#: C# Dev Kit, .NET Install Tool, GitLens та GitHub Copilot

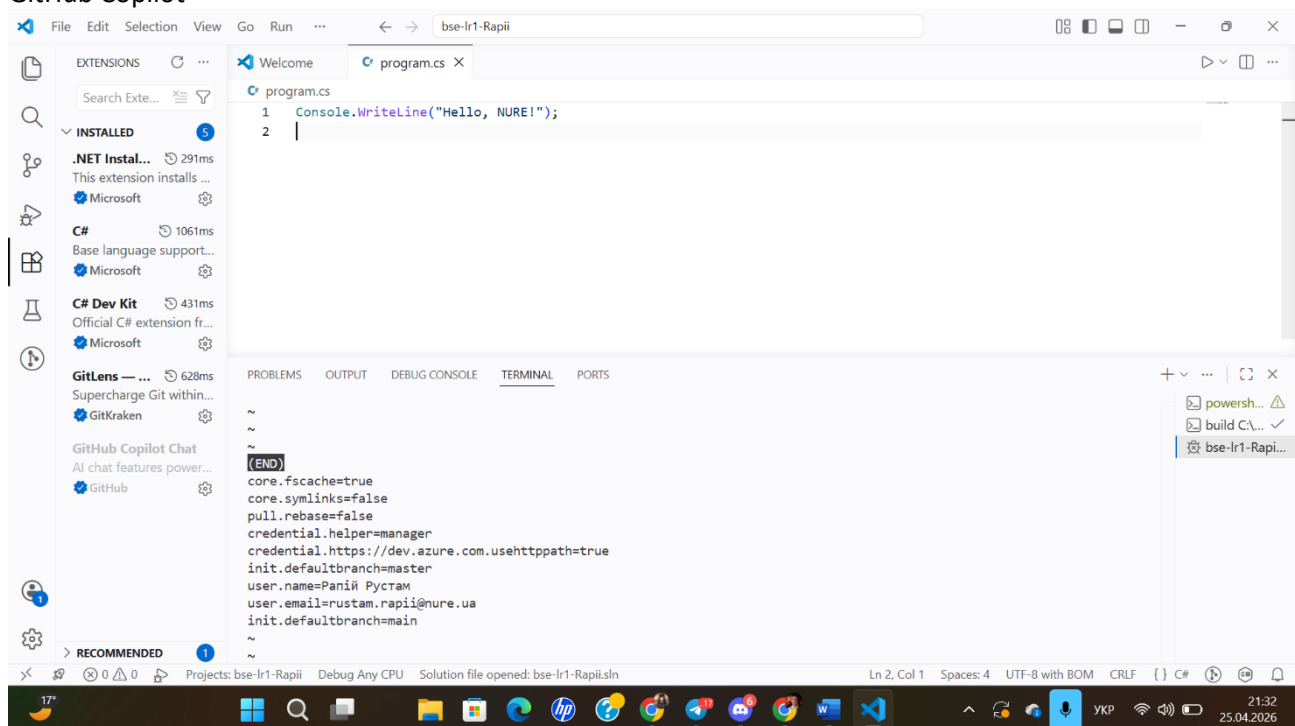


Рисунок 4.2 — Конфігурація глобальних параметрів Git: налаштування імені користувача та електронної пошти в домені @nure.ua.

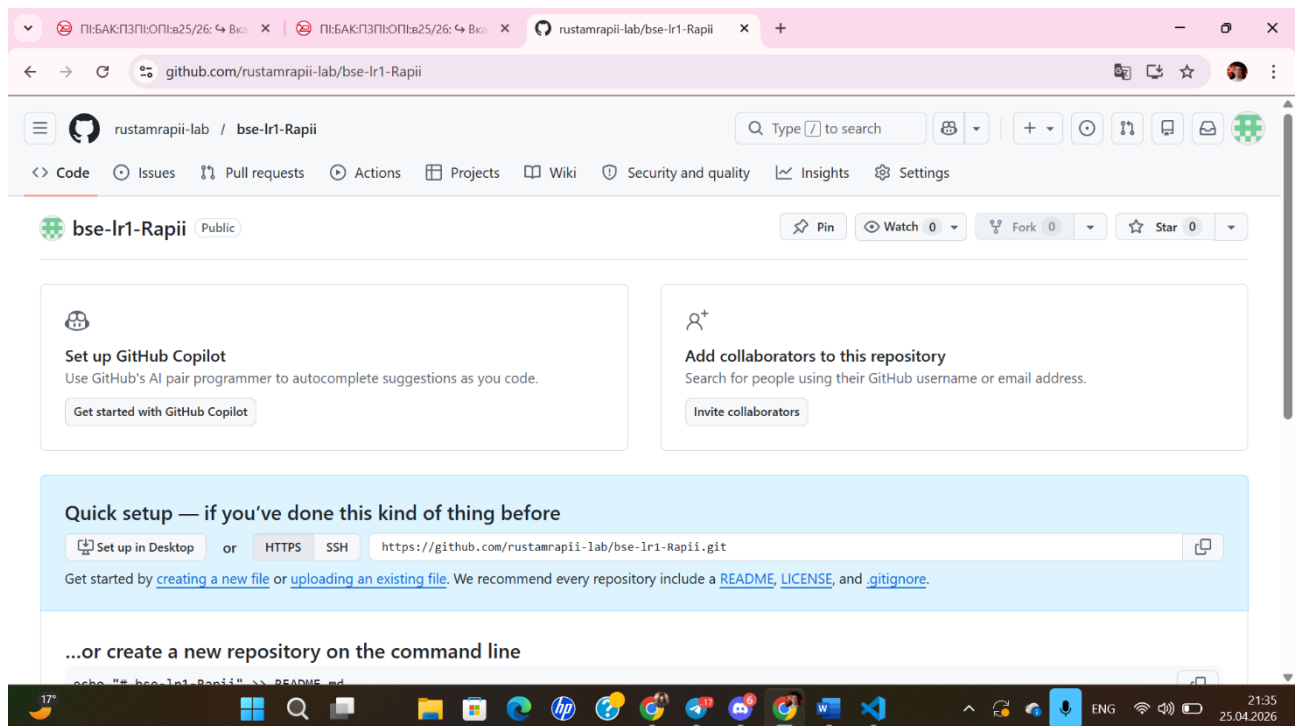


Рисунок 4.3 — Створення публічного репозиторію проекту на платформі GitHub

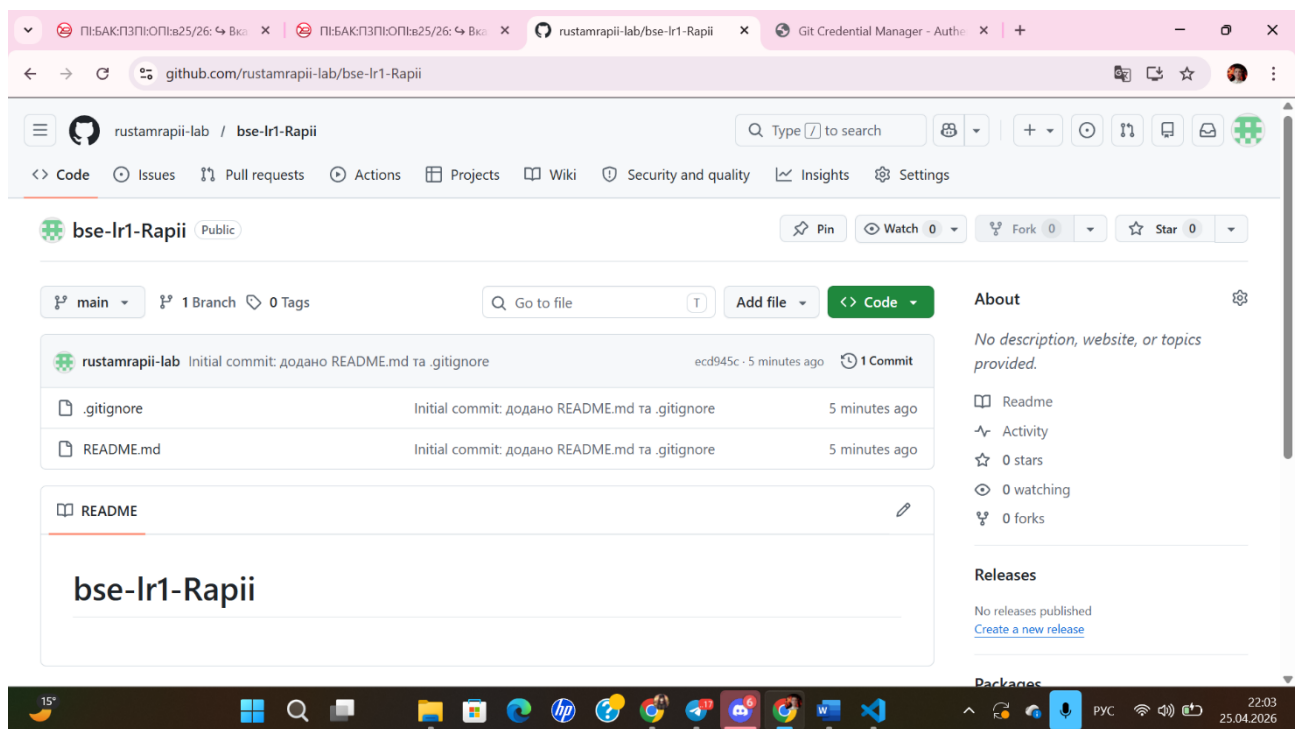


Рисунок 4.4 — Початковий стан репозиторію на GitHub після першого коміту: додано файли .gitignore та README.md.

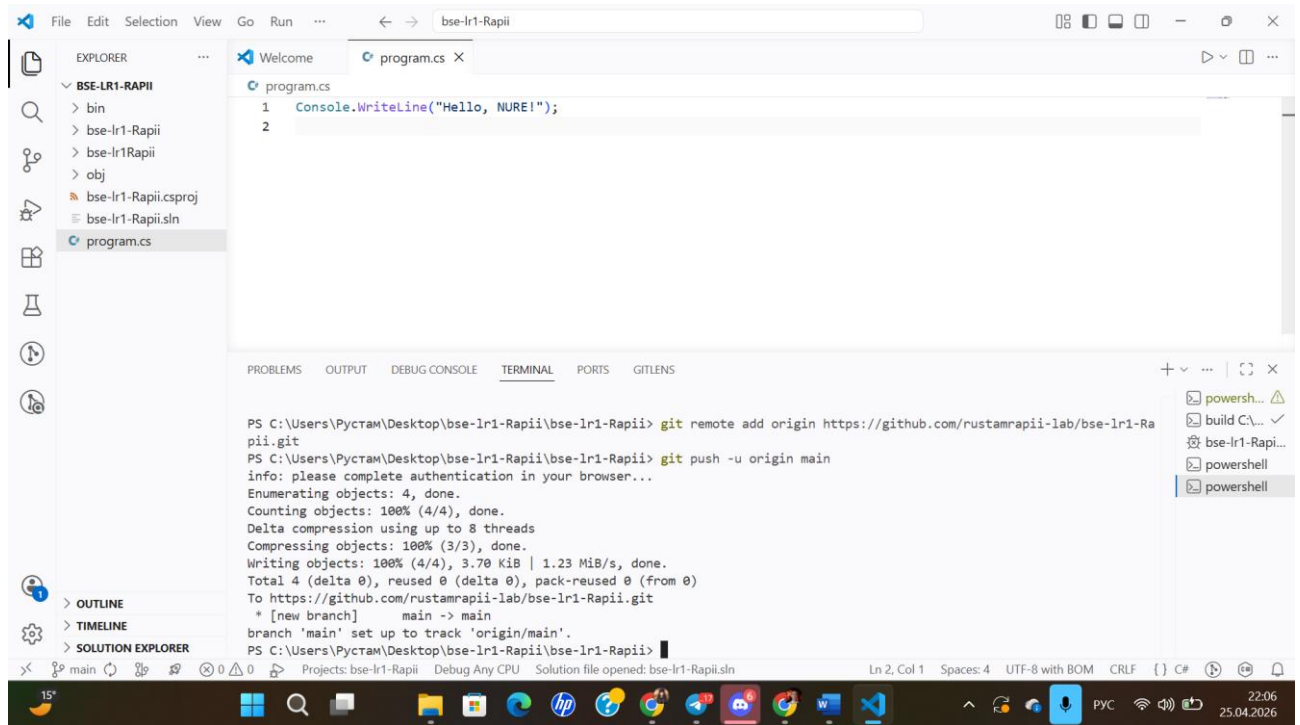


Рисунок 4.5 — Налаштування зв'язку з віддаленим репозиторієм та перша відправка (push) коду в гілку main.

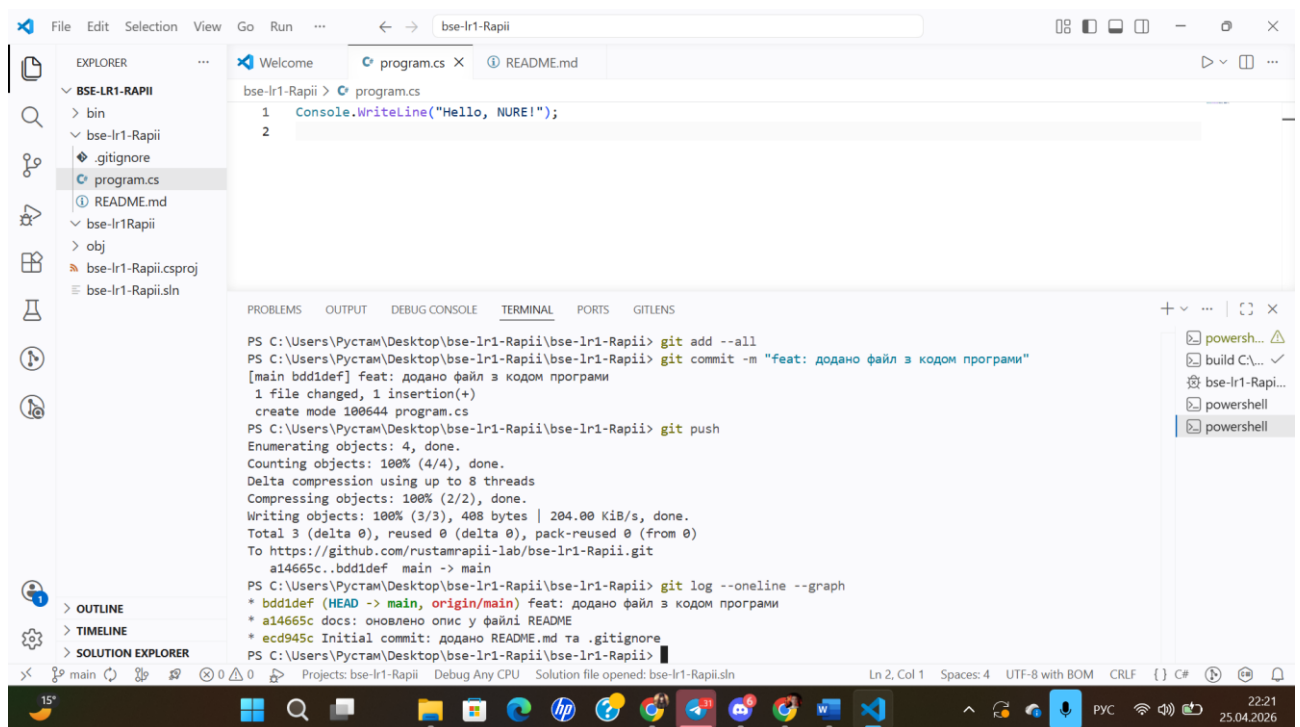


Рисунок 4.6 — Додавання файлу з кодом програми, виконання коміту та перевірка історії змін за допомогою команди git log.

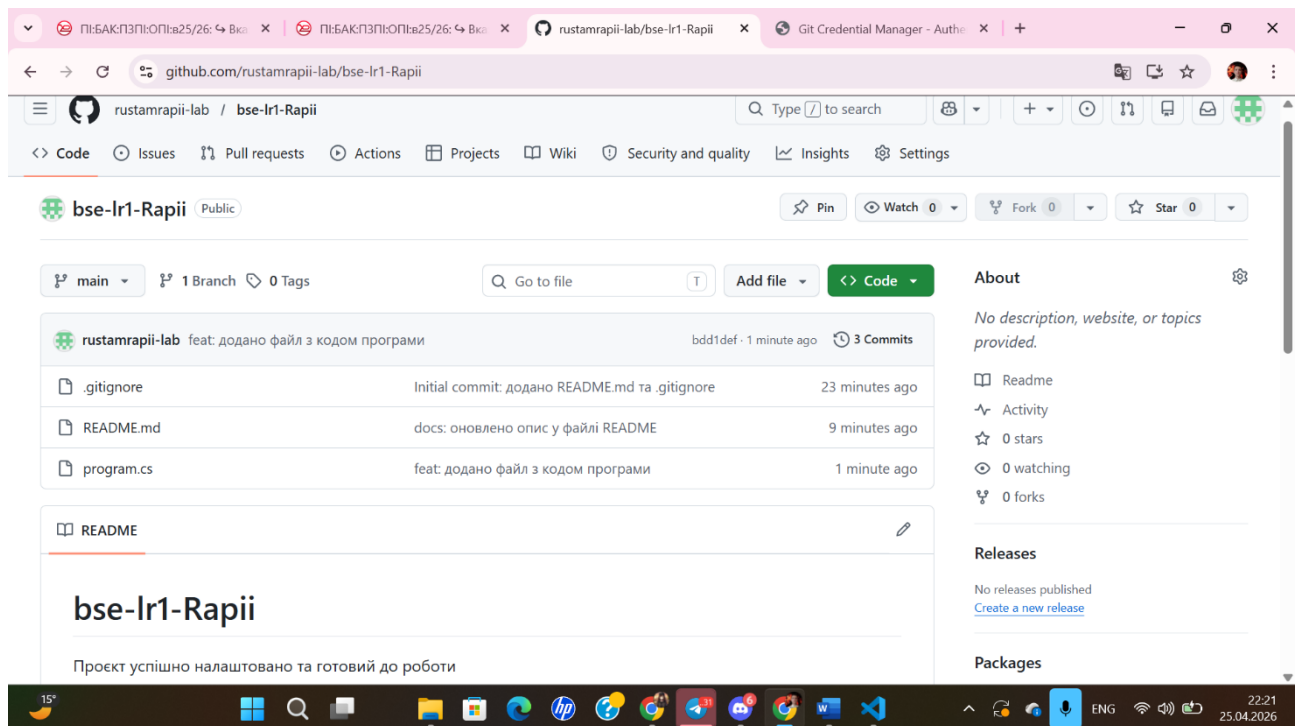


Рисунок 4.7 — Відображення структури репозиторію на GitHub після синхронізації: додано файл програми `program.cs` та оновлено документацію `README`.

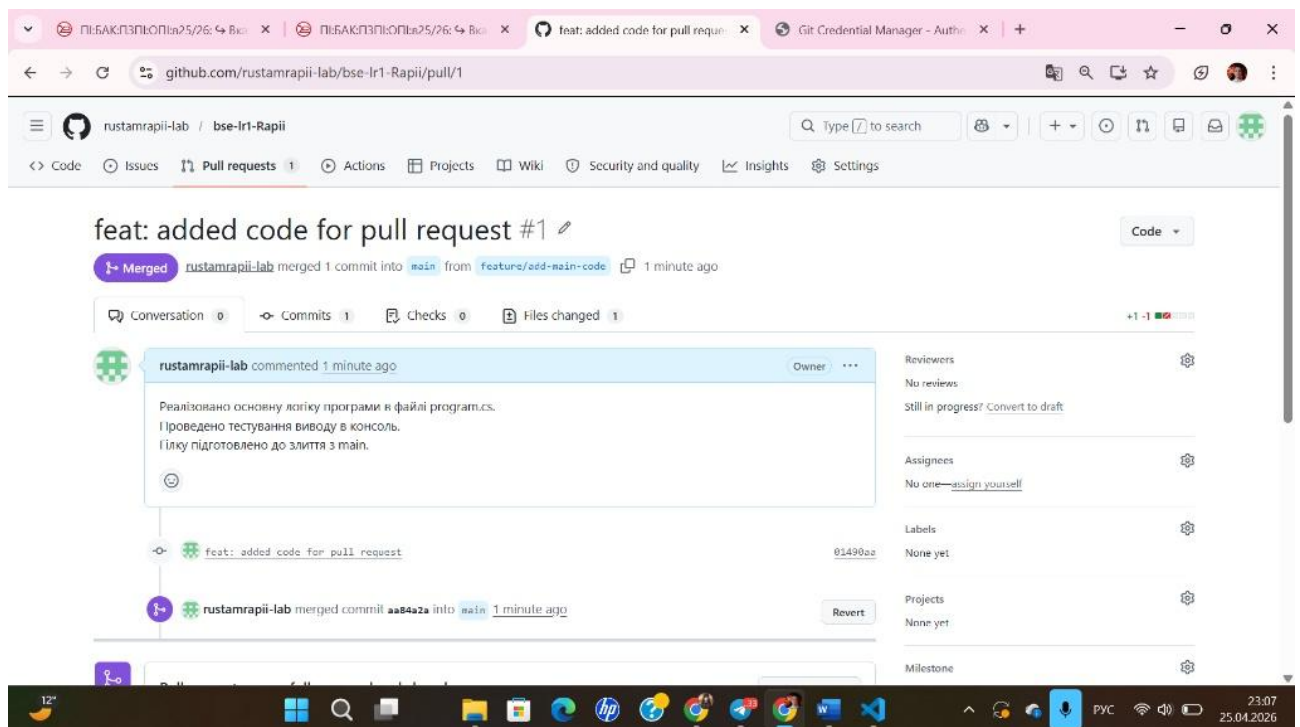


Рисунок 4.8 — Створення та успішне завершення Pull Request: злиття гілки `feature/add-main-code` у головну гілку `main` із описом виконаних робіт.

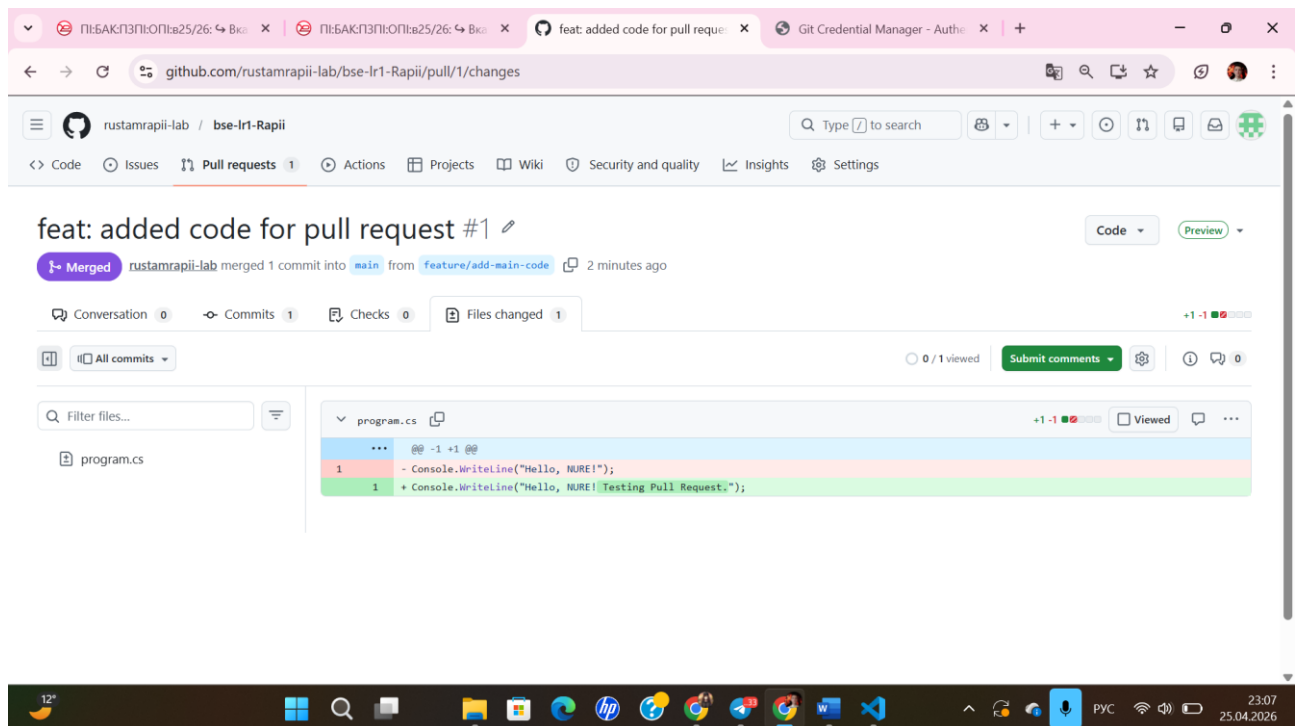


Рисунок 4.9 — Перегляд внесених змін (Diff) у межах Pull Request: демонстрація доданого коду для тестування функціоналу в програмі.

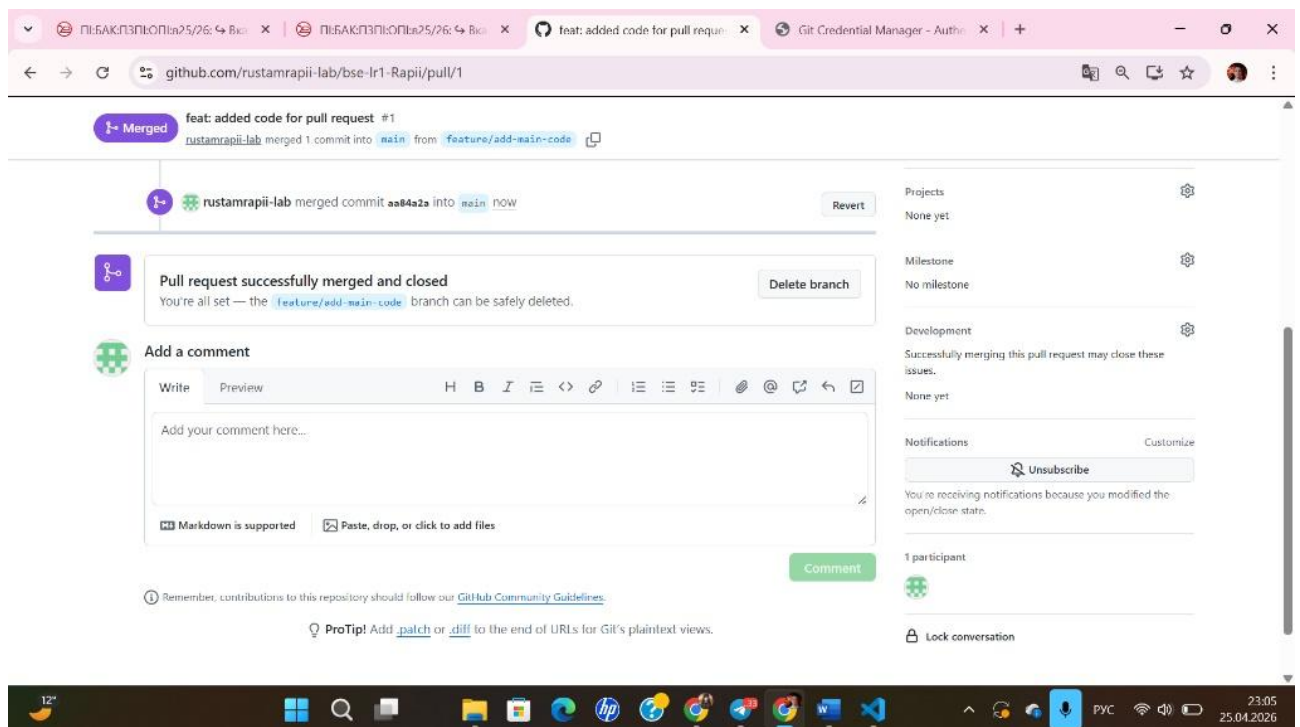


Рисунок 4.10 — Підтвердження успішного злиття Pull Request та закриття завдання в інтерфейсі GitHub.

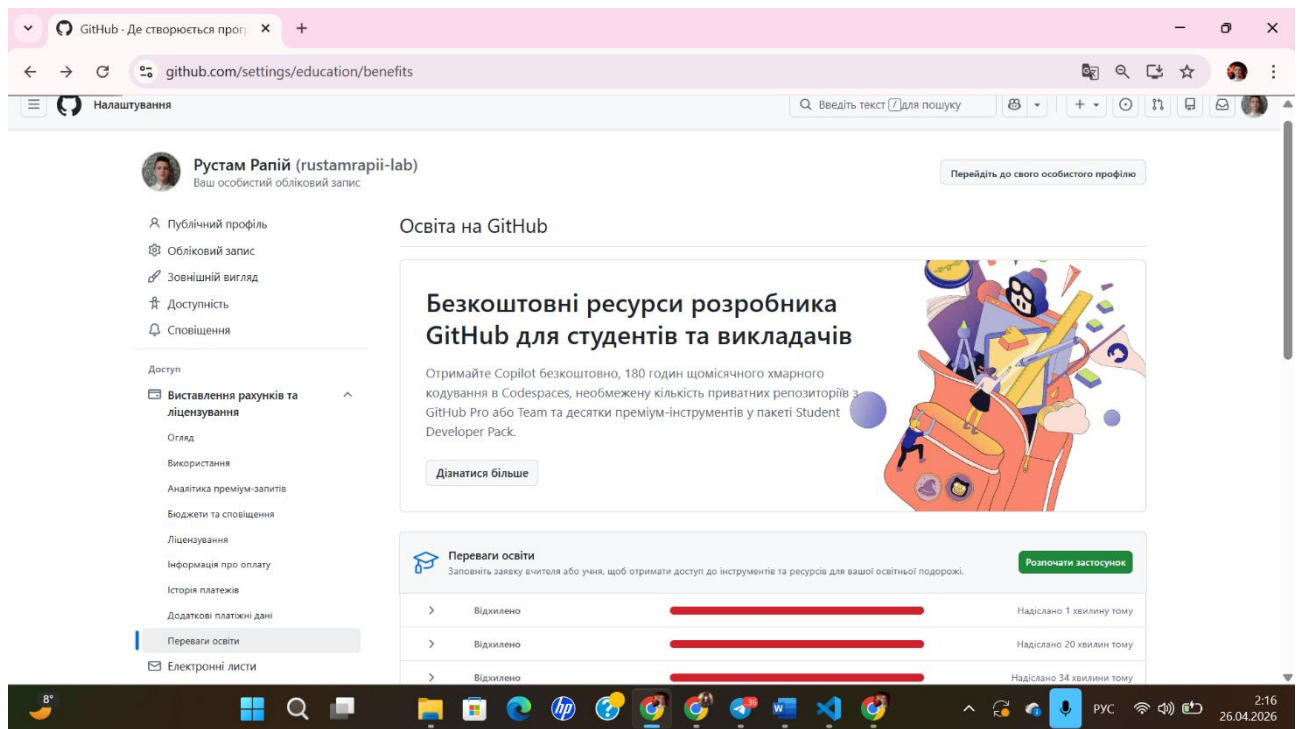


Рисунок 4.11 — Спроба активації безкоштовного доступу до GitHub Copilot через програму GitHub Student Developer Pack: відмова системи у верифікації статусу студента.

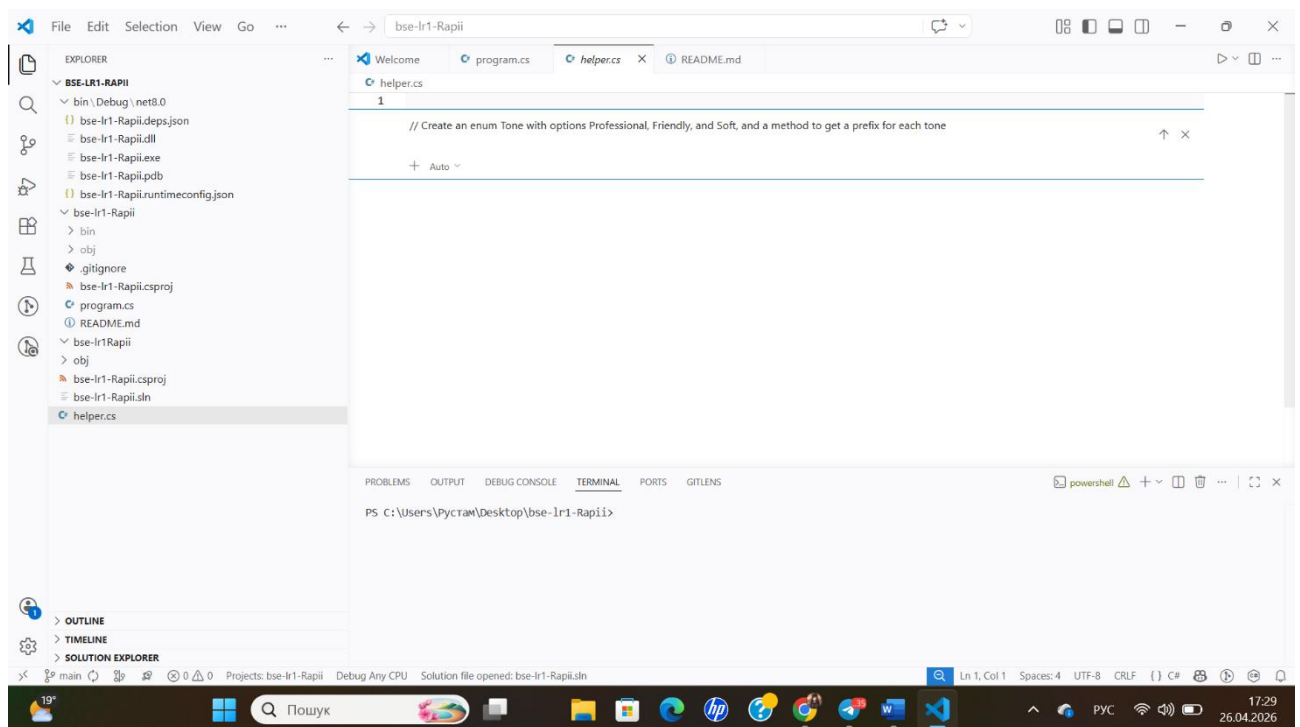


Рисунок 4.12 — Використання AI-асистента для генерації коду на мові C#: створення переліку (enum) Tone та методу обробки текстових префіксів.

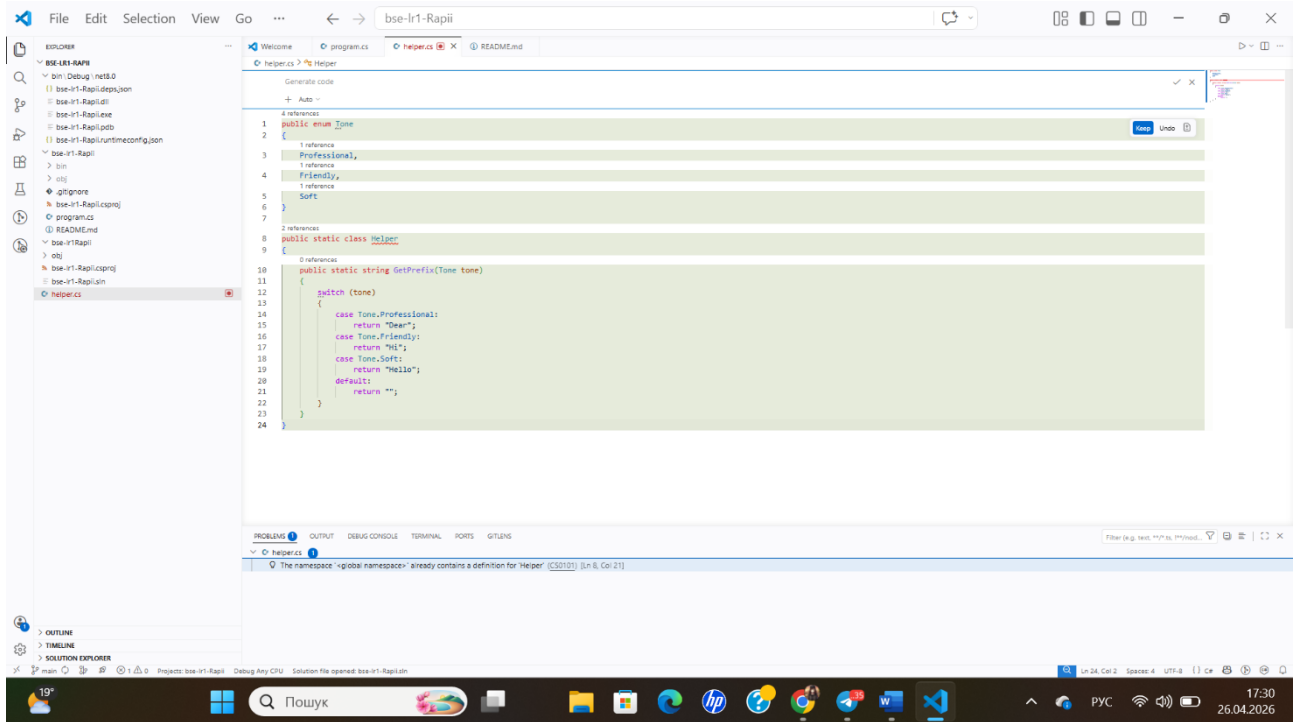


Рисунок 4.13 — Результат роботи AI-асистента: згенерований C# код з переліком Tone та методом GetPrefix, інтегрований у файл helper.cs.

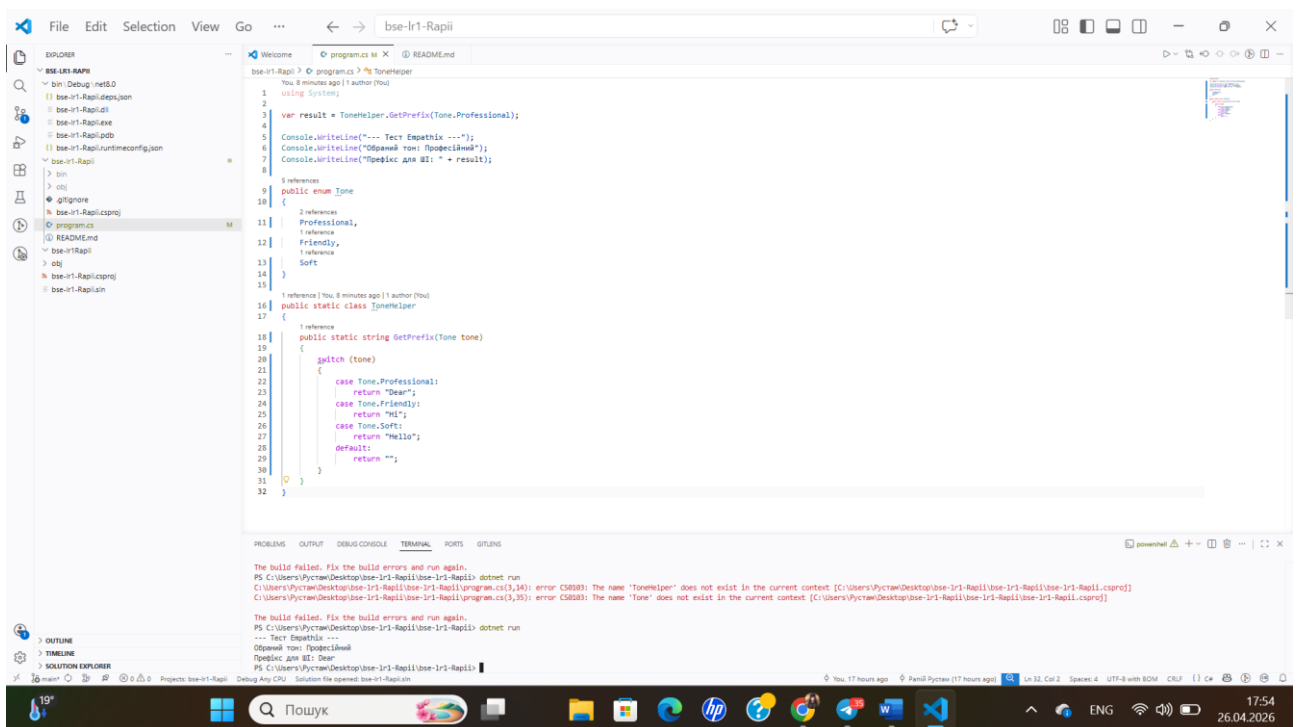


Рисунок 4.14 — Компіляція та запуск проекту за допомогою команди dotnet run: перевірка працездатності згенерованої логіки у терміналі VS Code.

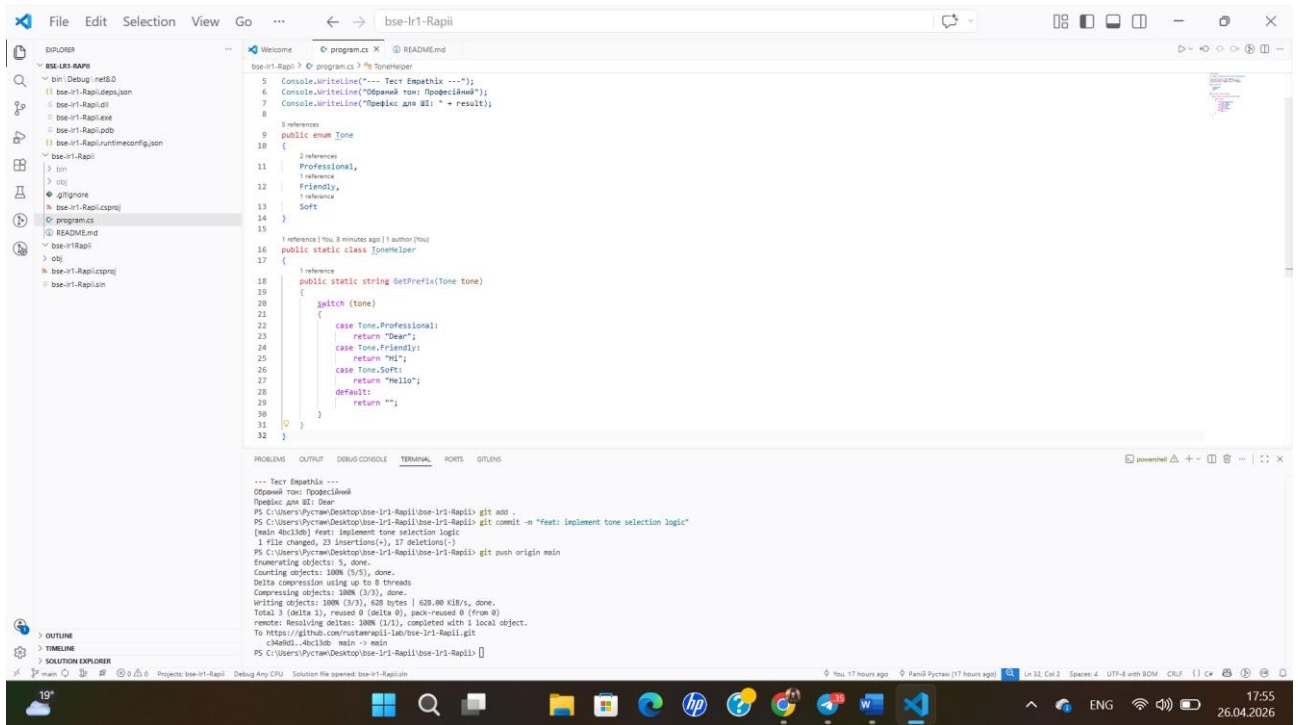


Рисунок 4.15 — Успішне виконання тесту програми та відправка (push) підтверджених змін із коментарем "feat: implement tone selection logic" до віддаленого репозиторію.

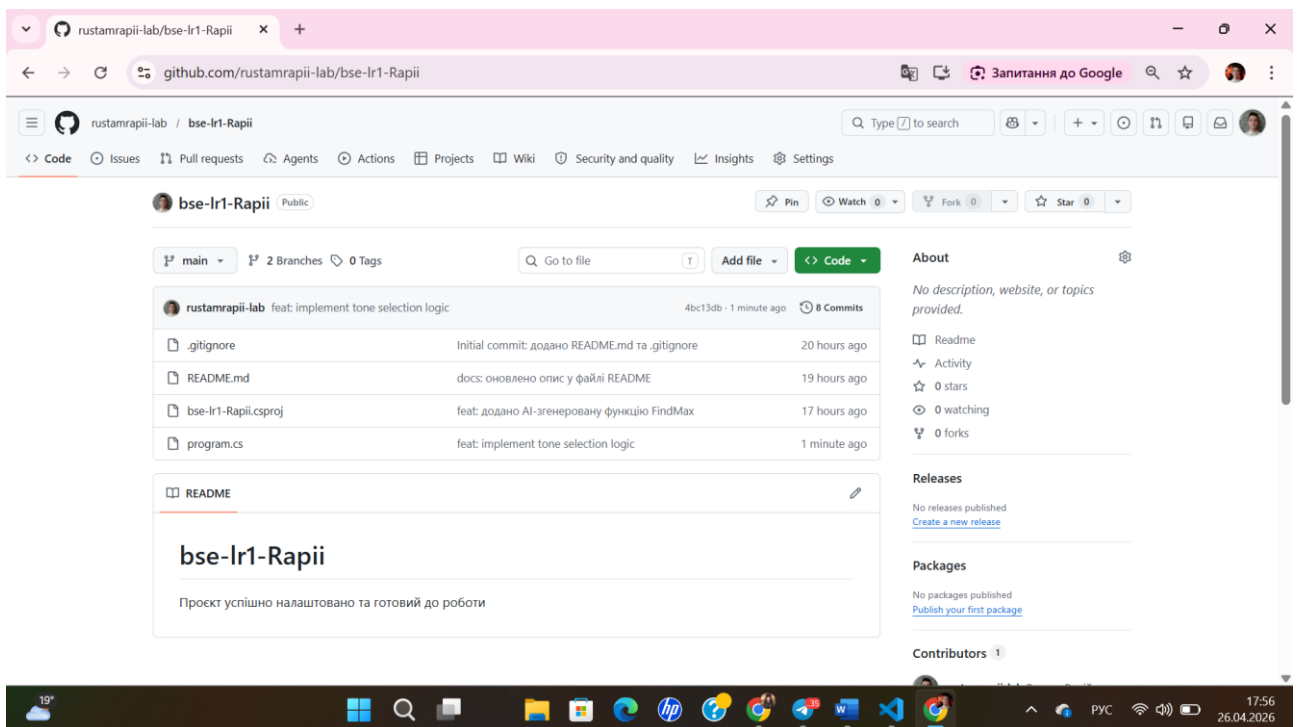


Рисунок 4.16 — Підсумковий вигляд структури репозиторію на GitHub: відображення фінальної історії комітів та всіх розроблених компонентів проекту.

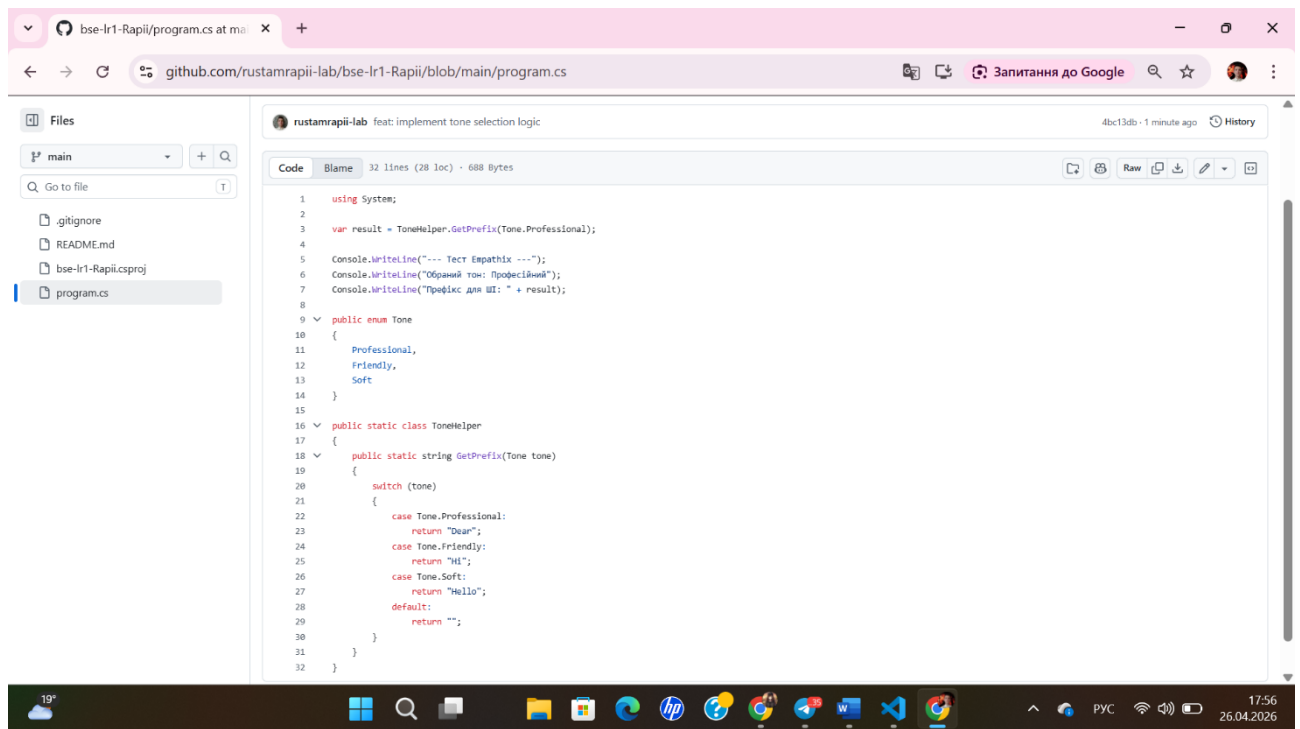


Рисунок 4.17 — Перегляд фінального вихідного коду файлу program.cs у віддаленому репозиторію GitHub після завершення всіх етапів розробки.

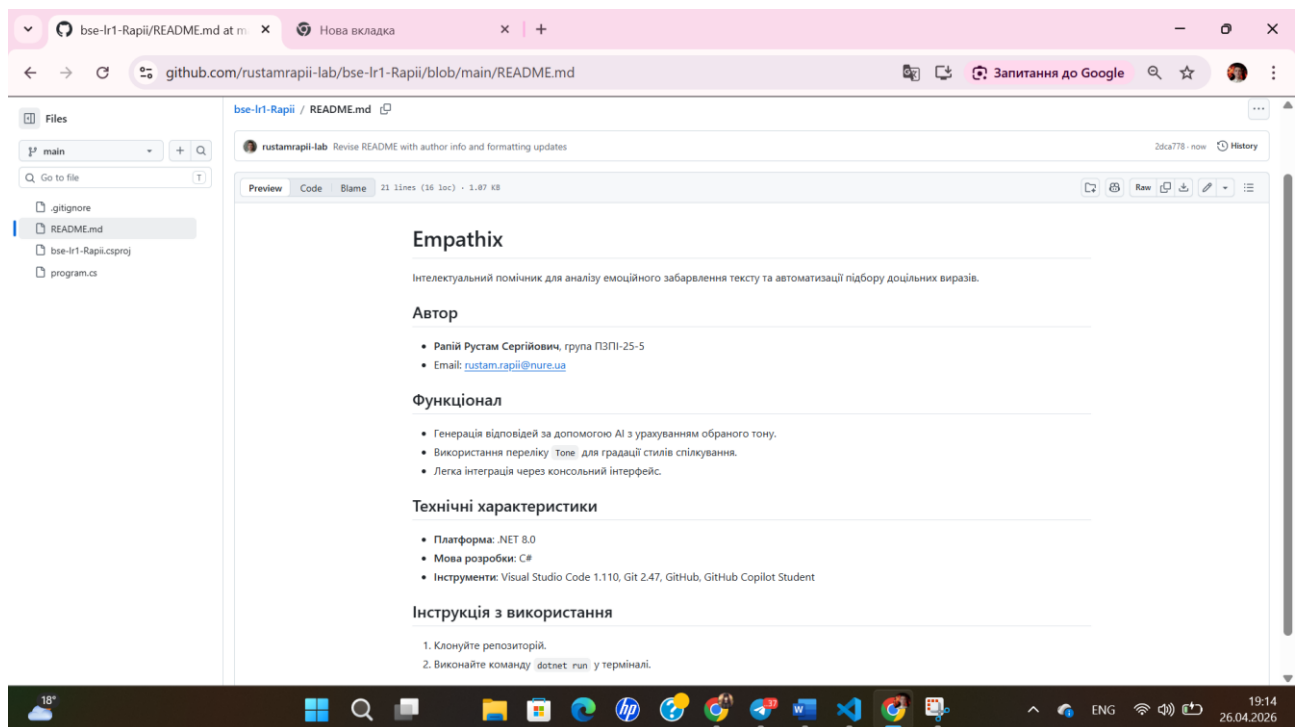
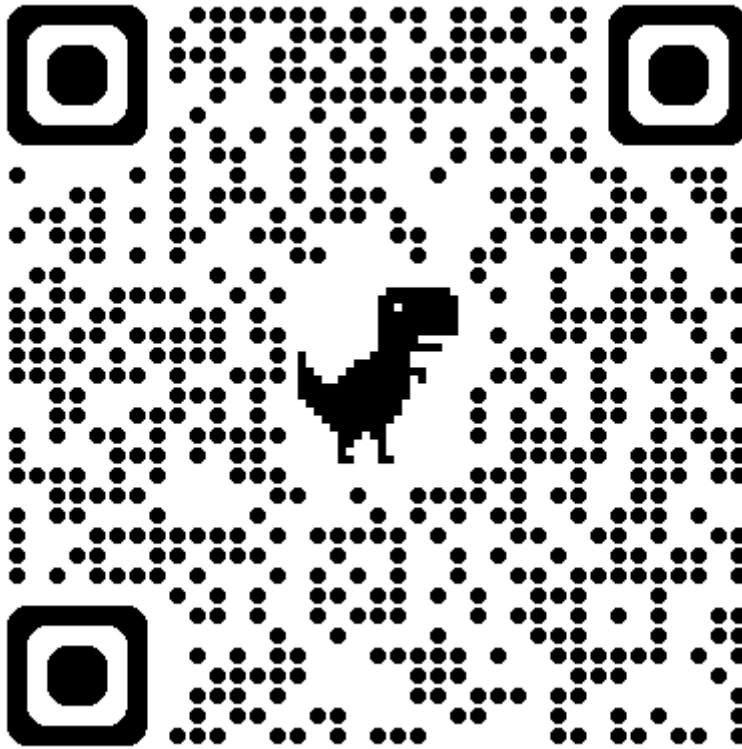


Рисунок 4.18 — Текст файлу README.md репозиторію проєкту "Empathix" на платформі GitHub

Проект "Empathix" розміщено на платформі GitHub за наступним посиланням:

<https://github.com/rustamrapii-lab/bse-lr1-Rapii>



ВИСНОВКИ

У ході виконання лабораторної роботи було опановано професійне інструментальне середовище розробника, що включає IDE Visual Studio Code, систему контролю версій Git та хмарну платформу GitHub. Було набуто практичних навичок з ініціалізації репозиторіїв, керування гілками та командної розробки через механізм Pull Request.

Конкретні результати:

- У ході роботи було створено локальний та віддалений репозиторії проєкту "**Empathix**".
- Загалом було виконано **8** комітів та використано **2** гілки (main та feature/add-main-code), які були успішно злиті.
- Налаштовано файл .gitignore для мови C# та оформлено документацію у форматі Markdown (файл README.md).

Досвід використання AI-асистента: Було здійснено спробу використання **GitHub Copilot** для генерації базових функцій проєкту. Досвід показав, що AI-асистент може бути корисним для швидкого створення логіки, проте на поточному етапі він вимагає високої уваги до деталей з боку розробника, оскільки запропоновані рішення часто містять помилки або вимагають рефакторингу.

Труднощі та їх подолання:

1. **Проблеми з верифікацією GitHub Student Developer Pack:** Під час спроби отримати студентську підписку виникли технічні труднощі з підтвердженням статусу навчання. Оскільки перевірка GitHub вимагає якісного фото документа, використання цифрового студентського квитка у застосунку Дія виявилось неможливим через відблиски екрана смартфона, які заважали системі розпізнати дані. Через відсутність фізичного пластикового квитка запит не був схвалений адміністрацією GitHub, тому для роботи використовувалися альтернативні методи генерації коду.
2. **Помилки компіляції згенерованого коду:** Початково згенерований AI файл `helper.cs` викликав помилки запуску через конфлікти структури проєкту та некоректне підключення просторів імен. Для вирішення проблеми було прийнято рішення оптимізувати архітектуру: файл `helper.cs` було видалено, а всю необхідну логіку перенесено безпосередньо до файлу `program.cs`.
3. **Ручне доопрацювання логіки:** Оскільки автоматично згенерований код був фрагментарним і не забезпечував повний цикл роботи програми, приклади використання та логіку вводу-виводу у файлі `program.cs` було дописано самостійно, що дозволило отримати стабільну роботу консольного додатка.